

**LAPORAN PRATIKUM STRUKTUR DATA
BINARY TREE DAN GRAPH DENGAN BAHASA JAVA**



Dosen Pengampu:
Wahyudi Dr.. S.T. M.T.

Disusun Oleh:
Kevin Maulana Sempri
2511531019

**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG
2026**

Kata Pengantar

Puji syukur penulis ucapkan kepada Allah SWT karena penulis dapat menyelesaikan laporan praktikum ini dengan judul "Binary Tree dan Graph Dengan Bahasa Java" dengan tepat waktu. Laporan ini dibuat sebagai salah satu untuk memenuhi nilai praktikum pada mata Struktur Data. Laporan ini berfokus pada studi kasus Pohon Biner dan Graf dengan menggunakan Bahasa Java. Penulis menyadari bahwa laporan praktikum ini masih jauh dari sempurna, oleh karena itu penulis mengucapkan banyak terima kasih kepada Bapak / Ibu dosen dan para asisten, serta rekan-rekan mahasiswa yang telah banyak membantu penulis, dalam memberikan arahan & ilmu. Untuk perbaikan & kemajuan di masa yang akan datang, penulis mengharapkan kritik & saran yang membangun. Semoga laporan ini bisa memberikan manfaat & wawasan bagi pembaca, khususnya pada studi kasus Pohon Biner dan Graf menggunakan Bahasa Java. Akhir kata, penulis ucapkan terima kasih kepada semuanya.

DAFTAR PUSTAKA

KATA PENGANTAR	i
DAFTAR ISI	ii
DAFTAR PUSTAKA	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan Pratikum.....	1
1.3 Manfaat Pratikum.....	1
BAB II HASIL DAN PEMBAHASAN.....	2
2.1 Waktu dan Tempat.....	2
2.2 Cara Kerja.....	2
2.3 Hasil.....	3
2.4 Pembahasan	7
DAFTAR PUSTAKA.....	

Bab 1 Pendahuluan

1.1 Latar Belakang

Perkembangan teknologi informasi menuntut adanya sistem penyimpanan data yang terstruktur dan mudah diakses. Struktur Data menjadi salah satu komponen penting dalam pengelolaan informasi, baik di bidang pendidikan, bisnis, maupun pemerintahan. Eclipse IDE merupakan salah satu perangkat lunak yang menyediakan fasilitas untuk merancang, mengelola, dan memanipulasi kode Bahasa pemrograman Java yang relatif mudah dipahami. Melalui praktikum ini, diharapkan mampu memahami Struktur Data Pohon Biner dan Struktur Data Graf serta mengimplementasikan studi kasus dengan menggunakan Bahasa Java.

1.2 Tujuan praktikum

1. Memahami konsep dasar struktur data dengan tipe data abstrak Binary Tree dan Graph.
2. Mempelajari cara membuat struktur data Binary Tree dan Graph dengan menggunakan Bahasa Java.
3. Mengimplementasikan struktur data sesuai kebutuhan studi kasus.
4. Melatih keterampilan dalam mengelola data secara sistematis dan efisien.

1.3 Manfaat praktikum

1. Memberikan pengalaman langsung dalam merancang dan mengelola struktur data.
2. Menambah wawasan tentang penggunaan struktur data dengan menggunakan Bahasa Java.
3. Mempersiapkan mahasiswa dengan keterampilan praktis yang dapat diterapkan dalam dunia kerja.
4. Membantu memahami pentingnya integritas dan konsistensi data dalam sistem informasi.

Bab 2 Hasil dan Pembahasan

Pada bab ini saya akan menampilkan hasil praktikum yang telah dilaksanakan beserta pembahasannya

2.1 Waktu dan Tempat

Praktikum ini dilaksanakan pada hari Senin tanggal 2 Juni 2026 Dilaksanakan di laboratorium Jurusan Informatika, Fakultas Teknologi Infomarsi, Universitas Andalas.

2.2 Cara Kerja

Praktikan akan membuat sebuah program yang berupa Bahasa Pemrograman Java. Perintahnya adalah membuat kode untuk Abstract Data Types (Tipe data abstrak) dan kode untuk Driver (memanggil ADT tersebut)

2.3 Hasil

Kode Java Node:

```
package pekan9_2511531019;

public class Node_2511531019 {
    int data_1019;
    Node_2511531019 left_1019;
    Node_2511531019 right_1019;

    public Node_2511531019(int data_1019) {
        this.data_1019 = data_1019;
        left_1019 = null;
        right_1019 = null;
    }

    public void setLeft_2511531019(Node_2511531019 node_1019) {
        if (left_1019 == null) {
            left_1019 = node_1019;
        }
    }

    public void setRight_2511531019(Node_2511531019 node_1019) {
        if (right_1019 == null) {
            right_1019 = node_1019;
        }
    }

    public Node_2511531019 getLeft_2511531019() {
        return left_1019;
    }
}
```

```

public Node_2511531019 getRight_2511531019() {
    return right_1019;
}

public int getData_2511531019() {
    return data_1019;
}

public void setData_2511531019(int data_1019) {
    this.data_1019 = data_1019;
}

void printPreorder_2511531019(Node_2511531019 node_1019) {
    if (node_1019 == null) {
        return;
    }
    System.out.print(node_1019.data_1019 + " ");
    printPreorder_2511531019(node_1019.left_1019);
    printPreorder_2511531019(node_1019.right_1019);
}

void printPostorder_2511531019(Node_2511531019 node_1019) {
    if (node_1019 == null) {
        return;
    }
    printPostorder_2511531019(node_1019.left_1019);
    printPostorder_2511531019(node_1019.right_1019);
    System.out.print(node_1019.data_1019 + " ");
}

void printInorder_2511531019(Node_2511531019 node_1019) {
    if (node_1019 == null) {
        return;
    }
    printInorder_2511531019(node_1019.left_1019);
    System.out.print(node_1019.data_1019 + " ");
    printInorder_2511531019(node_1019.right_1019);
}

public String print_2511531019() {
    return this.print_2511531019("", true, "");
}

public String print_2511531019(String prefix_1019, boolean
isTail_1019, String sb_1019) {
    if (right_1019 != null) {
        right_1019.print_2511531019(prefix_1019 +
(isTail_1019 ? "| ": " "), false, sb_1019);
    }
    System.out.println(prefix_1019 + (isTail_1019 ? "\\-- "
: "/-- ") + data_1019);
    if (left_1019 != null) {
        left_1019.print_2511531019(prefix_1019 +
(isTail_1019 ? "| ": " "), true, sb_1019);
    }
}

```

```

        return sb_1019;
    }
}

```

Kode Java BTree:

```

package pekan9_2511531019;

public class BTree_2511531019 {
    private Node_2511531019 root_1019;
    private Node_2511531019 currentNode_1019;
    public BTree_2511531019() {
        root_1019 = null;
    }
    public boolean search_2511531019(int data_1019) {
        return search_2511531019(root_1019, data_1019);
    }

    private boolean search_2511531019(Node_2511531019 node_1019, int
data_1019) {
        if (node_1019.getData_2511531019() == data_1019) {
            return true;
        }
        if (node_1019.getLeft_2511531019() != null) {
            if (search_2511531019(node_1019.getLeft_2511531019(),
data_1019)) {
                return true;
            }
        }
        if (node_1019.getRight_2511531019() != null) {
            if (search_2511531019(node_1019.getRight_2511531019(),
data_1019)) {
                return true;
            }
        }
        return false;
    }

    public void printInorder_2511531019() {
        root_1019.printInorder_2511531019(root_1019);
    }
    public void printPreorder_2511531019() {
        root_1019.printPreorder_2511531019(root_1019);
    }
    public void printPostorder_2511531019() {
        root_1019.printPostorder_2511531019(root_1019);
    }

    public Node_2511531019 getRoot_2511531019() {
        return root_1019;
    }

    public boolean isEmpty_2511531019() {
        return root_1019 == null;
    }
}

```

```

    }

    public int countNodes_2511531019() {
        return countNodes_2511531019(root_1019);
    }

    private int countNodes_2511531019(Node_2511531019 node_1019) {
        int count_1019 = 1;
        if (node_1019 == null) {
            return 0;
        } else {
            count_1019 +=
countNodes_2511531019(node_1019.getLeft_2511531019());
            count_1019 +=
countNodes_2511531019(node_1019.getRight_2511531019());
            return count_1019;
        }
    }

    public void print_2511531019() {
        root_1019.print_2511531019();
    }

    public Node_2511531019 getCurrent_2511531019() {
        return currentNode_1019;
    }

    public void setCurrent_2511531019(Node_2511531019 node_1019) {
        this.currentNode_1019 = node_1019;
    }

    public void setRoot_2511531019(Node_2511531019 root_1019) {
        this.root_1019 = root_1019;
    }
}

```

Kode Java BTreeDriver:

```

package pekan9_2511531019;
public class BTreeDriver_2511531019 {
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        // Membuat pohon
        BTree_2511531019 tree_1019 = new BTree_2511531019();
        System.out.print("Jumlah simpul awal pohon: ");
        System.out.println(tree_1019.countNodes_2511531019());

        //menambahkan simpul data 1
        Node_2511531019 root_1019 = new Node_2511531019(1);
        //menjadikan simpul 1 sebagai root
        tree_1019.setRoot_2511531019(root_1019);
        System.out.println("Jumlah simpul jika hanya ada root:");
        System.out.println(tree_1019.countNodes_2511531019());
        Node_2511531019 node2_1019 = new Node_2511531019(2);
        Node_2511531019 node3_1019 = new Node_2511531019(3);
        Node_2511531019 node4_1019 = new Node_2511531019(4);
    }
}

```



```

        Node_2511531019 node5_1019 = new Node_2511531019(5);
        Node_2511531019 node6_1019 = new Node_2511531019(6);
        Node_2511531019 node7_1019 = new Node_2511531019(7);
        Node_2511531019 node8_1019 = new Node_2511531019(8);
        Node_2511531019 node9_1019 = new Node_2511531019(9);
        root_1019.setLeft_2511531019(node2_1019);
        node2_1019.setLeft_2511531019(node4_1019);
        node2_1019.setRight_2511531019(node5_1019);
        node4_1019.setRight_2511531019(node8_1019);
        root_1019.setRight_2511531019(node3_1019);
        node3_1019.setLeft_2511531019(node6_1019);
        node3_1019.setRight_2511531019(node7_1019);
        node6_1019.setLeft_2511531019(node9_1019);

        //set root
        tree_1019.setCurrent_2511531019(tree_1019.getRoot_2511531019());
        System.out.println("Menampilkan simpul terakhir: ");

        System.out.println(tree_1019.getCurrent_2511531019().getData_2511531019());
        System.out.println("Jumlah simpul; setelah simpul 7
        ditambahkan");
        System.out.println(tree_1019.countNodes_2511531019());
        System.out.println("InOrder: ");
        tree_1019.printInorder_2511531019();
        System.out.println("\nPreOrder: ");
        tree_1019.printPreorder_2511531019();
        System.out.println("\nPostOrder: ");
        tree_1019.printPostorder_2511531019();
        System.out.println("\nMenampilkan simpul dalam bentuk
        pohon");
        tree_1019.print_2511531019();
    }
}

```

Kode Java GraphTraversal:

```

package pekan9_2511531019;
import java.util.*;
public class GraphTraversal_2511531019 {
    private Map<String, List<String>> graph = new HashMap<>();

    // Menambahkan edge (graf tak berarah)
    public void addEdge_2511531019(String node1_1019, String node2_1019) {
        graph.putIfAbsent(node1_1019, new ArrayList<>());
        graph.putIfAbsent(node2_1019, new ArrayList<>());
        graph.get(node1_1019).add(node2_1019);
        graph.get(node2_1019).add(node1_1019);
    }

    // Menampilkan graf awal
    public void printGraph_2511531019() {
        System.out.println("Graf Awal (Adjacency List):");
        for (String node_1019 : graph.keySet()) {
            System.out.print(node_1019 + ": ");
            List<String> neighbors = graph.get(node_1019);
            System.out.println(String.join(", ", neighbors));
        }
    }
}

```

```

    }
    System.out.println();
}

// DFS rekursif
public void dfs_2511531019(String start_1019) {
    Set<String> visited_1019 = new HashSet<>();
    System.out.println("Penelusuran DFS:");
    dfsHelper_2511531019(start_1019, visited_1019);
    System.out.println();
}

private void dfsHelper_2511531019(String current_1019, Set<String>
visited_1019) {
    if (visited_1019.contains(current_1019)) return;
    visited_1019.add(current_1019);
    System.out.print(current_1019 + " ");
    for (String neighbor : graph.getDefault(current_1019, new
ArrayList<>())) {
        dfsHelper_2511531019(neighbor, visited_1019);
    }
}

// BFS Iteratif
public void bfs_2511531019(String start) {
    Set<String> visited_1019 = new HashSet<>();
    Queue<String> queue_1019 = new LinkedList<>();
    queue_1019.add(start);
    visited_1019.add(start);
    System.out.println("Penelusuran BFS:");
    while (!queue_1019.isEmpty()) {
        String current_1019 = queue_1019.poll();
        System.out.print(current_1019 + " ");
        for (String neighbor_1019 : graph.getDefault(current_1019,
new ArrayList<>())) {
            if (!visited_1019.contains(neighbor_1019)) {
                queue_1019.add(neighbor_1019);
                visited_1019.add(neighbor_1019);
            }
        }
    }
    System.out.println();
}

// Main
public static void main(String[] args) {
    GraphTraversal_2511531019 graph_1019 = new
GraphTraversal_2511531019();

    // Contoh graf: A-B, A-C, B-D, B-E
    graph_1019.addEdge_2511531019("A", "B");
    graph_1019.addEdge_2511531019("A", "C");
    graph_1019.addEdge_2511531019("B", "D");
    graph_1019.addEdge_2511531019("B", "E");

    // Cetak graf awal
    System.out.println("Graf Awal adalah ");
}

```

```

graph_1019.printGraph_2511531019();

// Lakukan penelusuran
graph_1019.dfs_2511531019("A");
graph_1019.bfs_2511531019("A");
    }
}

```

Output Kode Java BTreeDriver:

```

Jumlah simpul awal pohon: 0
Jumlah simpul jika hanya ada root:
1
Menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
PreOrder:
1 2 4 8 5 3 6 9 7
PostOrder:
8 4 5 2 9 6 7 3 1
Dmenampilkan simpul dalam bentuk pohon
|  /-- 7
| /-- 3
|  \-- 6
|  | \-- 9
|-- 1
|  | /-- 5
|  | \-- 2
|  | | /-- 8
|  | | \-- 4

```

Output Kode Java GraphTraversal:

```

Graf Awal adalah
Graf Awal (Adjacency List):
A: B, C
B: A, D, E
C: A
D: B
E: B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E

```

2.4 Pembahasan

Praktikum ini menunjukkan bahwa Bahasa Java memudahkan proses membuat struktur data yang terdiri dari selector, mutator, dan lainnya. Dengan demikian, praktikum ini berhasil memberikan gambaran nyata tentang pentingnya Tipe Data Abstrak untuk struktur data dan driver sebagai eksekutor Tipe Data Abstrak untuk mendukung sistem informasi.

Bab 3 Kesimpulan

Praktikum ini berhasil menghasilkan struktur dalam data dengan menggunakan bahasa Java. Di dalam data memiliki struktur yang dapat menjalankan sebuah kasus kecil maupun besar untuk mendukung system informasi.